🔹 C++ `vector` – Full Explanation with Examples

A **vector** is a **dynamic array** in C++ (from STL) that can **grow and shrink automatically**.

```
#include <vector>
using namespace std;
```

🔹 1. Initialization (Creating a Vector)

✅ `vector<type> v;`

Creates an empty vector

```
vector<int> v;
```

- Size = 0
- Capacity = 0
- **Time:** O(1)

✅ `vector<type> v(N);`

Creates a vector with N elements (default initialized)

```
vector<int> v(5);
```

- Output: `0 0 0 0 0`
- **Time:** O(N)

✅ `vector<type> v(N, V);`

Creates N elements with value V

```
vector<int> v(5, 10);
```

- Output: `10 10 10 10 10`
- **Time:** O(N)

✅ `vector<type> v(v2);`

Copy another vector

```
vector<int> v2 = {1, 2, 3};
vector<int> v(v2);
```

- v = `1 2 3`
- **Time:** O(N)

✅ `vector<type> v(A, A+N);`

Copy from array

```
int arr[] = {5, 6, 7};
vector<int> v(arr, arr+3);
```

- v = `5 6 7`
- **Time:** O(N)

🔹 2. Capacity Functions

✅ `v.size()`

Returns number of elements

```
v.size();
```

✅ `v.max_size()`

Maximum elements vector can theoretically hold

```
v.max_size();
```

(OS / memory dependent)

✅ `v.capacity()`

Allocated memory size

```
v.capacity();
```

📌 Capacity ≥ Size

✅ `v.clear()`

Remove all elements (memory not freed)

```
v.clear();
```

- Size becomes 0
- Capacity remains
- **Time:** O(N)

✅ `v.empty()`

Check if vector is empty

```
if(v.empty())
    cout << "Empty";
```

✅ `v.resize(new_size)`

Change vector size

```
vector<int> v = {1,2,3};
v.resize(5);
```

- Result: `1 2 3 0 0`

```
v.resize(2);
```

- Result: `1 2`
- **Time:** O(K)

🔹 3. Modifiers (Modify Vector)

✅ `v = v2` or `v.assign()`

Assign another vector

```
vector<int> v1 = {1,2};
vector<int> v2 = {5,6,7};

v1 = v2;
```

- v1 = `5 6 7`

```
v.assign(3, 9);
```

- v = `9 9 9`

✅ `v.push_back(x)`

Add element at end

```
v.push_back(10);
```

📌 Amortized **O(1)**

✅ `v.pop_back()`

Remove last element

```
v.pop_back();
```

✅ `v.insert(pos, value)`

Insert at specific position

```
vector<int> v = {1,2,4};
v.insert(v.begin()+2, 3);
```

- Result: `1 2 3 4`
- **Time:** O(N)

✅ `v.erase(pos)`

Delete element

```
v.erase(v.begin()+1);
```

- Result: `1 3 4`

```
v.erase(v.begin(), v.begin()+2);
```

- Removes first 2 elements

🔹 4. Algorithm Functions (Not Vector Member)

✅ `replace()`

Replace values

```
replace(v.begin(), v.end(), 10, 99);
```

- Replace all `10` → `99`
- **O(N)**

✅ `find()`

Find an element

```
auto it = find(v.begin(), v.end(), 5);

if(it != v.end())
    cout << "Found";
```

🔹 5. Element Access

✅ `v[i]`

Fast access (no bounds check)

```
cout << v[2];
```

⚠️ Unsafe if index invalid

✅ `v.at(i)`

Safe access (bounds check)

```
cout << v.at(2);
```

Throws exception if invalid

✅ `v.front()`

First element

```
v.front();
```

✅ `v.back()`

Last element

```
v.back();
```

🔹 6. Iterators

✅ `v.begin()`

Points to first element

```
auto it = v.begin();
```

✅ `v.end()`

Points just after last element

```
for(auto it = v.begin(); it != v.end(); it++)
    cout << *it << " ";
```

🔹 Complete Example Program

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {1,2,3};

    v.push_back(4);
    v.insert(v.begin()+1, 10);

    for(int x : v)
        cout << x << " ";

    return 0;
}
```

Output

```
1 10 2 3 4
```

🔹 When to Use `vector`?

✅ Dynamic size needed
✅ Fast random access
✅ STL compatibility
✅ Competitive programming

If you want next:

- 🔥 **Vector interview questions**
- 🧠 **Memory & capacity deep explanation**

- 🧪 **Practice problems**
- ⚔️ **Vector vs Array vs List**

Just tell me 👍

Below is a **clear, exam-ready + interview-ready comparison** of **Vector vs Array vs List (C++)**, with **examples**, **use-cases**, and **when to choose what**.

🔥 Vector vs Array vs List (C++ STL)

🔹 Basic Definition

Structure	What it is
Array	Fixed-size contiguous memory
Vector	Dynamic array (resizable)
List	Doubly linked list

🔹 Memory Layout

Structure	Memory Type
Array	Contiguous
Vector	Contiguous
List	Non-contiguous (nodes + pointers)

📌 **Important:**
Contiguous memory ⇒ fast access
Non-contiguous ⇒ slow access, fast insert/delete

🔹 Size Behavior

Feature	Array	Vector	List
Size change	❌ No	✅ Yes	✅ Yes
Dynamic resize	❌	✅	✅

🔹 Time Complexity Comparison

Operation	Array	Vector	List
Access by index	O(1)	O(1)	❌ O(N)
Insert at end	❌	O(1)*	O(1)
Insert at middle	O(N)	O(N)	O(1)*
Delete at middle	O(N)	O(N)	O(1)*
Search	O(N)	O(N)	O(N)

*Using iterator position

🔹 Syntax & Example

🔹 Array Example

```
int arr[5] = {1,2,3,4,5};
cout << arr[2];   // Fast access
```

❌ Size fixed
❌ No built-in functions

🔹 Vector Example

```
vector<int> v = {1,2,3};
v.push_back(4);
cout << v[2];
```

✅ Dynamic
✅ STL support
❌ Slow insert in middle

🔹 List Example

```
list<int> l = {1,2,3};
l.push_front(0);
```

```
l.push_back(4);
```

✅ Fast insert/delete anywhere
❌ No direct indexing

🔹 Memory Overhead

Structure	Extra Memory
Array	❌ No
Vector	⚠️ Some (capacity)
List	❌❌ High (2 pointers per node)

🔹 Iterator Support

Structure	Iterator
Array	❌ Limited
Vector	✅ Random access
List	✅ Bidirectional

🔹 Real-World Use Cases

✅ Use Array when:

- Size is fixed
- Low-level / embedded systems
- Maximum performance required

📌 Example:
Matrix, static buffers

✅ Use Vector when:

- Size changes dynamically

- Fast access needed
- Most common use

📌 Example:
Dynamic data storage, competitive programming

✅ Use List when:

- Frequent insert/delete in middle
- No random access needed

📌 Example:
Undo/Redo operations, playlist navigation

🔹 One-Line Interview Answer 💡

> **Array** → Fixed size, fastest, no flexibility
> **Vector** → Dynamic array, balanced, most used
> **List** → Linked list, slow access, fast insert/delete

🔹 Visual Comparison

```
Array / Vector:
[1][2][3][4]

List:
[1] -> [2] -> [3] -> [4]
```

🔹 Which One Should You Choose?

Scenario	Best Choice
Random access needed	Vector
Fixed size	Array
Frequent mid insert/delete	List

If you want next:

- ⚔️ **Vector vs Deque vs List**
- 🧠 **Why vector is faster than list**
- 🎯 **MCQ & interview Q&A**
- 🧪 **Practice problems**

Just tell me 👍
Below is a **high-level, advanced STL interview Q&A set** — these are the kind of questions asked in **product-based companies, senior rounds, and system-level C++ interviews**.
I'll keep answers **clear, crisp, and technically correct**.

🔥 Advanced C++ STL – Interview Q&A

1️⃣ What is STL and how is it designed internally?

Answer:
STL consists of **four core components**:

1. **Containers** – data storage (`vector`, `map`)
2. **Algorithms** – operations (`sort`, `find`)
3. **Iterators** – glue between containers & algorithms
4. **Functors / Lambdas** – behavior customization

📌 STL follows **generic programming** using **templates**.

2️⃣ Why are STL algorithms container-independent?

Answer:
Because algorithms operate on **iterators**, not containers.

```
sort(v.begin(), v.end());
```

👉 Any container with **valid iterator category** works.

3️⃣ Explain iterator categories with examples.

Iterator	Containers
Input	istream
Output	ostream
Forward	forward_list
Bidirectional	list, set, map
Random Access	vector, deque

📌 `sort()` requires **random access iterator**

4️⃣ Why can't we sort a `list` using `std::sort`?

Answer:
`std::sort` needs **random access iterators**
`list` provides only **bidirectional iterators**

✅ Correct way:

```
l.sort();
```

5️⃣ Difference between `map` and `unordered_map` (internals)?

Feature	map	unordered_map
Structure	Red-Black Tree	Hash Table
Order	Sorted	Unordered
Search	O(log N)	O(1) avg
Memory	Less	More

6️⃣ What is amortized complexity?

Answer:
Average time per operation over a sequence of operations.

📌 Example:

- `vector.push_back()` → O(1) amortized

- Occasional O(N) reallocation

7️⃣ Why vector reallocation invalidates iterators?

Answer:

- Vector allocates **new memory**
- Copies elements
- Old memory destroyed

👉 All pointers & iterators become invalid

8️⃣ How does `reserve()` improve performance?

```
v.reserve(1000);
```

Answer:

- Prevents repeated reallocation
- Improves push_back speed
- Avoids iterator invalidation

9️⃣ Difference between `reserve()` and `resize()`?

Function	Effect
reserve()	Allocates memory
resize()	Changes size + initializes elements

🔟 What is a functor and why is it used?

Answer:
A **function object** (class with `operator()`).

```
struct cmp {
  bool operator()(int a, int b) {
    return a > b;
```

```
    }
};
```

Used in:

- `priority_queue`
- `set`
- `sort`

1️⃣1️⃣ When would you prefer lambda over functor?

Answer:

- Short logic
- Local usage
- Cleaner syntax

```
sort(v.begin(), v.end(), [](int a, int b){
    return a > b;
});
```

1️⃣2️⃣ Explain `emplace_back()` vs `push_back()`

Feature	push_back	emplace_back
Object creation	Outside	In-place
Performance	Slower	Faster

```
v.emplace_back(10, 20);
```

1️⃣3️⃣ Why unordered_map may degrade to O(N)?

Answer:

- Poor hash function
- Many collisions
- All keys in same bucket

1️⃣4️⃣ What is load factor in unordered_map?

```
load_factor = size / bucket_count
```

📌 High load factor → collisions

1️⃣5️⃣ How does `std::set` maintain order?

Answer:
Uses **Red-Black Tree**

- Self-balancing BST
- Guarantees O(log N)

1️⃣6️⃣ Why is `deque` faster than vector in front insertion?

Answer:

- Deque uses **multiple memory blocks**
- No shifting required at front

1️⃣7️⃣ Can STL containers be thread-safe?

Answer:
❌ No (by default)

✔️ External synchronization required (mutex)

1️⃣8️⃣ What is SFINAE in STL?

Answer:
Substitution Failure Is Not An Error

- Used in template selection
- Enables compile-time function filtering

1️⃣9️⃣ What happens if you erase while iterating a vector?

❌ Wrong:

```
for(auto it : v)
    v.erase(it);
```

✔️ Correct:

```
for(auto it = v.begin(); it != v.end(); )
{
    if(*it == 0) it = v.erase(it);
    else ++it;
}
```

2️⃣0️⃣ Difference between `stable_sort` and `sort`?

Feature	sort	stable_sort
Stability	❌ No	✅ Yes
Speed	Faster	Slower
Memory	Less	More

🔥 Rapid-Fire One-Liners

- STL uses **templates**
- Algorithms work on **iterators**
- `vector` is cache-friendly
- `map` is tree-based
- `unordered_map` is hash-based
- `list` uses linked nodes

🎯 Final Interview Tip

> If you can explain **WHY**, not just **WHAT**, you pass advanced rounds.

If you want next:

- 🧠 **STL internal implementation deep dive**
- ⚔️ **map vs unordered_map MCQs**
- 🧪 **Tricky STL coding questions**
- 📘 **Company-wise STL interview prep**

Just tell me 🚀